

uni-app壁纸预览：滑动提交评分与接口对接实现

在壁纸预览场景中，通过“滑动提交评分”的交互方式（如右滑点赞、左滑差评），能降低用户操作成本，提升评分参与度。本文将基于现有Swiper壁纸预览框架，新增滑动评分交互模块，完成与后端评分接口的对接，实现“滑动触发→状态反馈→接口提交→结果同步”的完整流程，并解决滑动冲突、重复提交等核心问题。

一、核心需求与交互设计

1. 核心需求

- 支持滑动交互提交评分：右滑提交“好评”（5分），左滑提交“中评”（3分），长按滑动提交“差评”（1分）；
- 对接后端评分接口，确保评分数据实时提交，支持重复评分时覆盖原有结果；
- 滑动过程中提供明确的视觉反馈（如滑动方向提示、评分等级预览）；
- 解决滑动评分与Swiper壁纸切换的手势冲突，确保两种交互独立可靠；
- 处理接口异常场景（如网络错误、请求超时），提供重试机制。

2. 交互设计规则

滑动方向	滑动距离要求	评分等级	视觉反馈
右滑（正向）	超过屏幕宽度1/4	好评（5分）	右侧出现绿色“好条”，伴随震动反馈
左滑（反向）	超过屏幕宽度1/4	中评（3分）	左侧出现黄色“中条”，伴随轻微震动
左滑长按	超过屏幕宽度1/2	差评（1分）	左侧出现红色“差条”，震动反馈增强
滑动距离不足	小于屏幕宽度1/4	无评分提交	页面回弹至初始状态反馈

二、接口对接规范与前置准备

1. 评分接口规范（示例）

假设后端提供的评分接口遵循RESTful规范，前端需按以下格式提交请求：

Code block

```
1  // 1. 接口地址
2  const RATING_API = "https://api.example.com/wallpaper/rating";
3
4  // 2. 请求方式: POST
5  // 3. 请求头
6  {
7    "Content-Type": "application/json",
8    "token": "用户登录令牌（从Storage获取）"
9  }
10
11 // 4. 请求参数
12 {
13   "wallpaperId": 123,      // 壁纸ID（必传）
14   "rating": 5,            // 评分（1/3/5，必传）
15   "userId": 45678         // 用户ID（必传，从登录信息获取）
16 }
17
18 // 5. 响应结果
19 // 成功响应（200）
20 {
21   "code": 200,
22   "msg": "评分成功",
23   "data": {
24     "wallpaperId": 123,
25     "rating": 5,
26     "updateTime": "2025-12-07 15:30:20"
27   }
28 }
29
30 // 失败响应（如400）
31 {
32   "code": 400,
33   "msg": "评分参数错误，请重试",
34   "data": null
35 }
```

2. 前置准备工作

1. **接口封装**：将评分接口封装为工具函数，统一处理请求头、参数格式与响应解析；

- 2. 权限校验：确保用户已登录（从Storage获取token），未登录时引导至登录页；
- 3. 状态管理：新增评分相关状态（如当前评分、评分状态、接口请求状态），与壁纸信息关联。

三、完整实现方案（含滑动交互与接口对接）

在现有壁纸预览页面中，新增滑动评分容器包裹Swiper组件，通过监听 touchstart、touchmove、touchend 事件实现滑动评分逻辑，同时通过手势判断规则解决与Swiper的交互冲突。

1. 页面结构（新增滑动评分容器与反馈模块）

Code block

```
1  <template>
2    <view class="wallpaper-preview">
3
4      <view class="top-info-bar" :style="{paddingTop: statusBarHeight + 'px'}">
5        <view class="tag-group">
6          <view class="tag" v-for="(tag, idx) in currentWallpaper.info.tags"
:key="idx">
7            {{ tag }}
8          </view>
9        </view>
10       <view class="resolution">
11         {{ currentWallpaper.info.resolution }}
12       </view>
13     </view>
14
15
16     <view
17       class="rating-swipe-container"
18       @touchstart="handleRatingTouchStart"
19       @touchmove="handleRatingTouchMove"
20       @touchend="handleRatingTouchEnd"
21       :style="{transform: `translateX(${swipeX}px)`, transition:
swipeTransition}"
22     >
23
24       <view class="rating-feedback left-feedback" :class="{ 'show':
showLeftFeedback}">
25         <view class="feedback-icon" :style="{backgroundColor:
leftFeedbackColor}">
26           <image
27             :src="leftFeedbackIcon"
28             mode="widthFix"
29             class="feedback-img"
```

```

30         ></image>
31     </view>
32     <view class="feedback-text" :style="{color: leftFeedbackColor}">
33         {{ leftFeedbackText }}
34     </view>
35 </view>
36
37
38     <view class="rating-feedback right-feedback" :class="{ 'show':
showRightFeedback}">
39         <view class="feedback-icon" style="backgroundColor: #4cd964">
40             <image
41                 src="/static/icons/good.png"
42                 mode="widthFix"
43                 class="feedback-img"
44             ></image>
45         </view>
46         <view class="feedback-text" style="color: #4cd964">
47             好评
48         </view>
49     </view>
50
51
52     <swiper
53         class="preview-swiper"
54         :current="currentIndex"
55         :duration="300"
56         :circular="false"
57         @change="handleSwiperChange"
58         :disable-touch="isRatingSwipe"
59     >
60         <swiper-item v-for="(item, idx) in wallpaperList" :key="idx">
61             <view class="wallpaper-container">
62                 <image
63                     class="wallpaper-img thumbnail"
64                     :src="item.thumbUrl"
65                     mode="widthFix"
66                     v-if="!item.hdLoaded"
67                 ></image>
68                 <image
69                     class="wallpaper-img hd-img"
70                     :src="item.url"
71                     mode="widthFix"
72                     @load="handleHdLoad(idx)"
73                     v-if="idx === currentIndex || item.hdLoaded"
74                     :style="{transform: `scale(${scale[idx]})`, opacity:
item.hdLoaded ? 1 : 0, transition: 'opacity 0.3s ease'}"

```

```

75         @tap="handleDoubleTap(idx)"
76     ></image>
77     <view class="loading-tips" v-if="idx === currentIndex &&
!item.hdLoaded">
78         <view class="loading-spinner"></view>
79         <text class="loading-text">加载高清图...</text>
80     </view>
81 </view>

```

2. 核心逻辑（滑动评分+接口对接）

Code block

```

1  <script>
2  import { saveImageToPhotosAlbum, setWallpaper } from '@/utils/wallpaper.js';
3  import { submitWallpaperRating } from '@/api/rating.js'; // 评分接口封装
4  import { getStorageSync } from '@/utils/storage.js'; // Storage工具函数
5
6  export default {
7    data() {
8      return {
9        wallpaperList: [], // 壁纸列表（含专属信息）
10       currentIndex: 0, // 当前壁纸索引
11       scale: [], // 壁纸缩放比例
12       statusBarHeight: 0, // 状态栏高度
13       collected: new Set(), // 已收藏壁纸ID
14
15       // 新增：滑动评分相关状态
16       startX: 0, // 触摸起始X坐标
17       startY: 0, // 触摸起始Y坐标
18       swipeX: 0, // 滑动偏移量（X轴）
19       swipeTransition: 'transform 0.2s ease', // 滑动过渡效果
20       screenWidth: 0, // 屏幕宽度（用于判断滑动距离）
21       isRatingSwipe: false, // 是否处于评分滑动中（用于禁用Swiper）
22       isLongPress: false, // 是否为长按滑动（用于判断差评）
23       longPressTimer: null, // 长按计时器
24
25       // 新增：评分反馈相关状态
26       showLeftFeedback: false, // 显示左侧反馈条
27       showRightFeedback: false, // 显示右侧反馈条
28       leftFeedbackColor: '#ffcc00', // 左侧反馈条颜色（中评-黄色）
29       leftFeedbackText: '中评', // 左侧反馈文本
30       leftFeedbackIcon: '/static/icons/middle.png', // 左侧反馈图标
31       showRatingToast: false, // 显示评分提示弹窗
32       ratingToastText: '', // 评分提示文本
33       toastBgColor: '#4cd964', // 提示弹窗背景色
34
35       // 新增：接口请求相关状态

```

```

36     isSubmitting: false,          // 是否正在提交评分（防重复提交）
37     loginToken: '',              // 用户登录令牌
38     userId: 0                    // 用户ID
39 };
40 },
41 computed: {
42     // 当前壁纸信息（含评分状态）
43     currentWallpaper() {
44         const wallpaper = this.wallpaperList[this.currentIndex] || {};
45         // 若未存储用户评分，默认返回空
46         if (!wallpaper.userRating) {
47             return { ...wallpaper, userRating: 0 };
48         }
49         return wallpaper;
50     }
51 },
52 onLoad(options) {
53     // 1. 接收壁纸列表与当前索引
54     const { index, wallpaperList } = options;
55     this.currentIndex = Number(index);
56     this.wallpaperList = JSON.parse(decodeURIComponent(wallpaperList)).map(item
=> ({
57         ...item,
58         hdLoaded: false,
59         userRating: 0 // 初始化用户评分
60     })));
61
62     // 2. 基础数据初始化
63     const systemInfo = uni.getSystemInfoSync();
64     this.statusBarHeight = systemInfo.statusBarHeight;
65     this.screenWidth = systemInfo.screenWidth;
66     this.scale = new Array(this.wallpaperList.length).fill(1);
67
68     // 3. 读取用户登录信息（用于评分接口）
69     const userInfo = getStorageSync('userInfo');
70     if (userInfo) {
71         this.loginToken = userInfo.token;
72         this.userId = userInfo.id;
73     } else {
74         // 未登录：引导至登录页
75         uni.navigateTo({ url: '/pages/login/index' });
76         return;
77     }
78
79     // 4. 读取已收藏状态与历史评分
80     const collectedIds = getStorageSync('wallpaper_collected');
81     if (collectedIds) this.collected = new Set(collectedIds);

```

```
82     const historyRatings = getStorageSync('wallpaper_ratings') || {};  
83     // 同步历史评分到壁纸列表  
84     this.wallpaperList.forEach(item => {  
85         if (historyRatings[item.id]) {  
86             item.userRating = historyRatings[item.id];  
87         }  
    }
```

3. 接口封装与样式设计

3.1 评分接口封装 (api/rating.js)

Code block

```
1  // api/rating.js  
2  import { request } from '@utils/request.js'; // 基础请求封装  
3  
4  /**  
5   * 提交壁纸评分  
6   * @param {Object} params - 评分参数 (wallpaperId, rating, userId)  
7   * @param {string} token - 用户登录令牌  
8   * @returns {Promise}  
9   */  
10 export const submitWallpaperRating = (params, token) => {  
11     return request({  
12         url: '/wallpaper/rating',  
13         method: 'POST',  
14         headers: {  
15             'Content-Type': 'application/json',  
16             'token': token  
17         },  
18         data: params  
19     });  
20 };  
21  
22 // 基础请求封装 (utils/request.js)  
23 export const request = (options) => {  
24     return new Promise((resolve, reject) => {  
25         uni.request({  
26             url: `https://api.example.com${options.url}`,  
27             method: options.method || 'GET',  
28             headers: options.headers || {},  
29             data: options.data || {},  
30             success: (res) => {  
31                 // 统一响应解析  
32                 if (res.statusCode === 200) {  
33                     resolve(res.data);  
34                 } else if (res.statusCode === 401) {
```

```
35         // 未授权 (token失效) , 引导重新登录
36         uni.navigateTo({ url: '/pages/login/index' });
37         reject(new Error('登录已过期, 请重新登录'));
38     } else {
39         reject(new Error(`请求失败: ${res.statusCode}`));
40     }
41 },
42 fail: (err) => {
43     reject(new Error(`网络异常: ${err.errMsg}`));
44 }
45 });
46 });
47 };
```

3.2 核心样式 (滑动评分与反馈)

Code block

```
1  <style scoped>
2  /* 原有样式省略, 新增以下内容 */
3
4  /* 滑动评分容器 */
5  .rating-swipe-container {
6      width: 100%;
7      height: 100%;
8      position: relative;
9      overflow: hidden;
10 }
11
12 /* 评分反馈条样式 */
13 .rating-feedback {
14     position: absolute;
15     top: 0;
16     height: 100%;
17     width: 120rpx;
18     display: flex;
19     flex-direction: column;
20     align-items: center;
21     justify-content: center;
22     opacity: 0;
23     transition: opacity 0.2s ease;
24     z-index: 5;
25 }
26 .rating-feedback.show {
27     opacity: 1;
28 }
```



```
29  .left-feedback {
30    left: 0;
31  }
32  .right-feedback {
33    right: 0;
34  }
35  .feedback-icon {
36    width: 80rpx;
37    height: 80rpx;
38    border-radius: 50%;
39    display: flex;
40    align-items: center;
41    justify-content: center;
42    margin-bottom: 20rpx;
43  }
44  .feedback-img {
45    width: 40rpx;
46    height: 40rpx;
47  }
48  .feedback-text {
49    font-size: 32rpx;
50    font-weight: bold;
51  }
52
53  /* 当前评分展示样式 */
54  .current-rating {
55    display: flex;
56    align-items: center;
57    gap: 10rpx;
58    margin-right: 20rpx;
59  }
60  .rating-label {
61    color: #fff;
62    font-size: 22rpx;
63  }
64  .rating-stars {
65    display: flex;
66    align-items: center;
67    gap: 4rpx;
68  }
69  .star-icon {
70    width: 24rpx;
71    height: 24rpx;
72  }
73
74  /* 评分提示弹窗样式 */
75  .rating-toast {
```

```
76     position: fixed;
77     top: 50%;
78     left: 50%;
79     transform: translate(-50%, -50%);
80     z-index: 100;
81     display: flex;
82     align-items: center;
83     justify-content: center;
84 }
85 .toast-content {
86     padding: 20rpx 30rpx;
87     border-radius: 16rpx;
88     background-color: #fff;
89     box-shadow: 0 4rpx 12rpx #ccc;
90 }
```

四、关键问题解决方案

1. 滑动交互冲突问题

核心矛盾：Swiper的滑动切换与评分滑动都依赖水平触摸事件，易出现交互混乱。解决方案：

- **滑动方向判断**：在 `touchmove` 事件中，通过对比X轴与Y轴的偏移量（`Math.abs(diffX) > Math.abs(diffY) * 2`），优先响应水平滑动为评分交互，垂直滑动为Swiper交互；
- **动态禁用Swiper**：当判断为评分滑动时，设置 `isRatingSwipe: true`，通过Swiper的 `disable-touch` 属性禁用其触摸响应，避免冲突；
- **滑动范围限制**：限制评分滑动的最大偏移量（右滑不超过屏幕1/3，左滑不超过1/2），避免滑动过度影响体验。

2. 重复提交与接口异常处理

1. **防重复提交**：新增 `isSubmitting` 状态，接口请求期间禁止再次触发提交逻辑；
2. **token失效处理**：在基础请求封装中监听401响应，引导用户重新登录并保存当前评分状态，登录后自动重试；
3. **网络异常重试**：评分失败时在提示弹窗中添加“重试”按钮，点击后重新调用 `submitRating` 方法；
4. **本地缓存备份**：接口请求失败时，先将评分结果存入本地缓存，待网络恢复后通过定时任务自动提交。

3. 用户体验优化

- **震动反馈**：在滑动达到评分阈值时，调用 `uni.vibrateShort()` 触发短震动，增强交互感知；
- **历史评分同步**：从Storage读取用户历史评分，进入预览页时自动同步到对应壁纸，避免重复评分；
- **滑动阻力感**：通过调整滑动过渡动画的缓动函数，模拟物理滑动的阻力感，提升交互真实度；

- **弱网提示：**检测到弱网时，在评分反馈中提示“弱网环境，评分已缓存”，降低用户焦虑。

五、总结

实现“滑动提交评分与接口对接”的核心，是通过精准的手势判断解决交互冲突，结合严谨的状态管理确保评分数据的可靠提交。本文方案不仅完成了滑动评分的基础交互与接口对接，还通过防重复提交、异常重试、历史同步等细节设计，提升了功能的稳定性与用户体验。实际开发中，可根据产品需求调整滑动规则（如评分等级、滑动距离阈值），或扩展“滑动评论”“滑动分享”等类似交互，进一步丰富壁纸预览功能的实用性。

（注：文档部分内容可能由 AI 生成）